

Bhasha / Language	Hinglish (Hindi + English)
Tools	VS Code + Python 3.10+ + OpenCV
Duration	2 hours per day
Prerequisite	Day 1 + Day 2 (Python Basics) done hona chahiye
Workshop	Smart Traffic System — Phase 1 (Days 1-4)

Contents

Day 3	OpenCV kya hai Install Image read/display Image properties BGR vs RGB Resize & Save
Day 4	Webcam live feed Drawing shapes Text on frame waitKey & exit Mini project

Teacher ke liye note

Yeh PDF screen pe open karke padhao — students VS Code mein code likhein.

Day 3 mein koi bhi image use kar sakte hain — ek traffic photo save karke rakhein.

Day 4 ke liye Jetson se connected camera chahiye, ya laptop webcam bhi kaam karega.

Har topic mein pehle analogy, phir code, phir ek chhota exercise hai.

Day 3

Images in OpenCV

OpenCV install | imread | imshow | Properties | BGR vs RGB | Resize & Save

Topic 1 — OpenCV kya hai?

Real life analogy

OpenCV = Computer ki aankhein.

Jaise hamaari aankhein duniya dekhti hain — rang, shape, movement —

OpenCV bhi computer ko wohi karne deta hai: images aur video samajhna.

Hamare traffic system mein: 4 cameras se video aayega, OpenCV use karega frames process karne ke liye.

Install karo — terminal mein ek baar:

VS Code terminal mein type karo

```
pip install opencv-python
```

Import karo aur version check karo:

```
import cv2
```

Version check karo

```
print('OpenCV version:', cv2.__version__)
```

Output / Terminal:

```
OpenCV version: 4.8.0
```

cv2 kyun? opencv nahi?

Library ka naam opencv-python hai lekin Python mein import cv2 likhte hain.

Yeh ek old naming convention hai — confuse mat hona!

Hamesha: import cv2 — yahi likhna hai.

Topic 2 — Image Read aur Display karna

Real life analogy

Sochlo tum apne phone se ek traffic junction ki photo lo.

cv2.imread() = photo memory mein load karna (file kholo)

cv2.imshow() = woh photo screen pe dikhana (dekho)

cv2.waitKey() = kitni der dikhani hai decide karna

cv2.destroyAllWindows() = window band karna (window close karo)

Code — apni koi bhi image use karo:

Koi bhi .jpg ya .png image VS Code folder mein rakh lo.

File ka naam neech code mein daalo. Hum yahan 'traffic.jpg' use kar rahe hain.

```
import cv2

# Image load karo (imread = image read)

img = cv2.imread('traffic.jpg')

# Check karo — image mili ya nahi?

if img is None:

    print('Image nahi mili! File ka naam check karo.')

else:

    print('Image successfully load hui!')

    cv2.imshow('Traffic Image', img)    # Window mein dikhao

    cv2.waitKey(0)                      # 0 = koi bhi key dabao tab tak rukho

    cv2.destroyAllWindows()            # Saari windows band karo
```

Output / Terminal:

```
Image successfully load hui!
(Ek naye window mein traffic.jpg dikhega)
(Koi bhi key dabao — window band ho jayegi)
```

Common galtiyan — dhyan rakhna!

img is None aata hai toh image file nahi mili — path check karo.

File ka naam exactly waise likhna jaise save hua hai — capital/small letters matter karte hain.

waitKey(0) zaroor likhna — warna window ek second bhi nahi rukegi!

destroyAllWindows() likhna mat bhoolo — warna windows band nahi hongi.

Try it yourself:

Apni koi bhi photo folder mein daalo.

Code mein file ka naam change karo.

Run karo — image dikhti hai?

Koi key dabao — window band hoti hai?

Topic 3 — Image Properties (shape, size, dtype)

Real life analogy

Image ek grid hoti hai — chote-chote colored squares (pixels) se bani.

shape = image kitni lambi (height) aur kitni chaudi (width) hai, aur kitne colour channels hain

size = total pixels kitne hain (height x width x channels)

dtype = har pixel ki value kis format mein store hai (uint8 = 0 to 255)

Code:

```
import cv2

img = cv2.imread('traffic.jpg')

# Properties dekho

print('Shape  :', img.shape)  # (height, width, channels)

print('Size   :', img.size)    # total pixels

print('Dtype  :', img.dtype)   # data type

# Alag alag access karo

height = img.shape[0]

width  = img.shape[1]
```

```
channels = img.shape[2]

print(f'Height: {height}px | Width: {width}px | Channels: {channels}')
```

Output / Terminal:

```
Shape : (480, 640, 3)
Size : 921600
Dtype : uint8
Height: 480px | Width: 640px | Channels: 3
```

Property	Kya batata hai?
<code>img.shape[0]</code>	Height — kitni rows hain (upar se neeche)
<code>img.shape[1]</code>	Width — kitne columns hain (left se right)
<code>img.shape[2]</code>	Channels — kitne colour layers hain (BGR = 3)
<code>img.size</code>	Total pixels = height x width x channels
<code>img.dtype</code>	uint8 matlab 0-255 values — normal image

Try it yourself:

Apni image load karo aur shape print karo.
Height aur width alag print karo — values kya hain?
Ek chhoti image aur ek badi image ka size compare karo.

Topic 4 — BGR vs RGB

Real life analogy

Sochlo traffic light ke 3 bulbs: Red, Green, Blue.
Normal duniya mein hum bolte hain RGB — Red pehle, Green doosra, Blue teesra.
OpenCV ulta sochta hai — BGR order mein store karta hai: Blue pehle, Green doosra, Red teesra.
Yeh confusing lagta hai, lekin har jagah yaad rakhna hai — colors galat na dikhe isiliye!

BGR aur RGB ka fark — visual samjho:

<code>OpenCV imread()</code>	BGR (Blue, Green, Red)	Default — hamesha aisa hi
<code>Matplotlib / web</code>	RGB (Red, Green, Blue)	Normal convention
<code>cv2.cvtColor()</code>	Conversion karna	BGR → RGB ya dono taraf
Traffic light	Red = (0, 0, 255) in BGR	Uta lagta hai lekin sahi hai!

Code — BGR se RGB convert karna:

```
import cv2

img_bgr = cv2.imread('traffic.jpg')    # BGR format mein load hoga

# BGR se RGB convert karo

img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Grayscale (black & white) banana

img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

cv2.imshow('Original (BGR)', img_bgr)

cv2.imshow('Grayscale',      img_gray)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Traffic system mein yeh kab kaam aayega?

YOLO detection BGR image expect karta hai — default OpenCV format theek hai.

Grayscale conversion se processing fast hoti hai — kam data process karna padta hai.

Jab bhi colors draw karein (bounding box, text) — BGR mein likhna hai!

Try it yourself:

traffic.jpg load karo.

BGR aur Grayscale dono windows mein dikhao.

Dono windows ek saath band karo — kya difference dikh raha hai?

Topic 5 — Image Resize aur Save karna

Real life analogy

Camera se badi image aati hai — 1920x1080 ya aur badi.

YOLO model ke liye ya fast processing ke liye hum image chhoti karni padti hai.

cv2.resize() = image ka size change karna

cv2.imwrite() = processed image file mein save karna

Code:

```
import cv2

img = cv2.imread('traffic.jpg')

print('Original size:', img.shape)

# Fixed size mein resize karo

img_small = cv2.resize(img, (320, 240))    # (width, height) – dhyan rakhna!

print('Resized size:', img_small.shape)

# Half size karo (percentage se)

h, w = img.shape[:2]

img_half = cv2.resize(img, (w // 2, h // 2))

print('Half size:', img_half.shape)

# Save karo

cv2.imwrite('traffic_small.jpg', img_small)

print('Saved: traffic_small.jpg')

cv2.imshow('Original', img)

cv2.imshow('Resized', img_small)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Output / Terminal:

```
Original size: (480, 640, 3)
Resized size:  (240, 320, 3)
Half size:      (240, 320, 3)
Saved: traffic_small.jpg
```

Zaroori — resize mein width pehle aata hai!

img.shape deta hai (height, width, channels) — height pehle
cv2.resize(img, (width, height)) — width pehle
Yeh ulta hai — confuse mat hona! resize mein hamesha (w, h) likhna.

Try it yourself:

Apni image ko 400x300 mein resize karo.
Resized image ko 'output.jpg' naam se save karo.
Folder mein jaao — nai file dikh rahi hai?

Mini Project: Image Inspector

Ab sab topics ek saath — image load, properties, grayscale, resize, save:

```
# Day 3 Mini Project — Image Inspector
```

```
import cv2
```

```
img = cv2.imread('traffic.jpg')
```

```
if img is None:
```

```
    print('Image nahi mili!')
```

```
else:
```

```
    # Properties print karo
```

```
    h, w, c = img.shape
```

```
    print(f'Image loaded: {w} x {h} pixels, {c} channels')
```

```
    print(f'Total pixels: {img.size}')
```


Grayscale banana

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Resize karo — YOLO ke liye standard size

```
resized = cv2.resize(img, (640, 480))
```

Save karo

```
cv2.imwrite('gray_traffic.jpg', gray)
```

```
cv2.imwrite('resized_traffic.jpg', resized)
```

```
print('Saved: gray_traffic.jpg aur resized_traffic.jpg')
```

Saari images dikhao

```
cv2.imshow('Original', img)
```

```
cv2.imshow('Grayscale', gray)
```

```
cv2.imshow('Resized', resized)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Ghar pe challenge (optional)

Grayscale image ka shape print karo — channels kitne hain? (1 ya 3?)

cv2.flip(img, 1) try karo — kya hota hai?

cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE) try karo.

Day 4

Webcam + Drawing on Frames

VideoCapture | Live feed loop | Shapes | Text | waitKey | Mini project

Day 3 ka Quick Recap

cv2.imread() — image file load karna

cv2.imshow() — image ya frame screen pe dikhana

img.shape — (height, width, channels) properties

BGR format — OpenCV mein Blue pehle aata hai, Red baad mein

cv2.resize() — image chhoti ya badi karna

Topic 1 — Webcam se Live Video Lena (VideoCapture)

Real life analogy

Sochlo ek security guard hai jo CCTV monitor dekhta rehta hai — continuously, ek ek frame.

VideoCapture(0) = camera kholo (0 = pehla camera / built-in webcam)

cap.read() = ek frame lo camera se

Yeh loop mein chalta hai — jab tak 'q' key na dabao, frames aate rehte hain.

Traffic system mein: yahi loop har frame mein YOLO detection chalayega.

Webcam ka simplest code:

```
import cv2

# Camera kholo (0 = default camera)

cap = cv2.VideoCapture(0)

# Camera properly khula ya nahi?

if not cap.isOpened():

    print('Camera nahi mila!')

else:

    print('Camera ready hai!')
```

```
# Ek frame lo aur dikhao

ret, frame = cap.read()      # ret = True/False, frame = image

if ret:

    cv2.imshow('My Camera', frame)

    cv2.waitKey(0)

cap.release()                # Camera band karo

cv2.destroyAllWindows()
```

Live loop — har frame dikhao:

```
import cv2

cap = cv2.VideoCapture(0)

while True:                  # Hamesha chalta raho

    ret, frame = cap.read()   # Naya frame lo

    if not ret:               # Frame nahi mila?

        print('Frame nahi aaya!')

        break

    cv2.imshow('Live Camera', frame) # Frame dikhao

    key = cv2.waitKey(1)      # 1ms wait karo

    if key == ord('q'):        # 'q' dabate hi band karo

        break

cap.release()                # Camera band karo
```

```
cv2.destroyAllWindows()
```

Output / Terminal:

```
(Live webcam window khulegi)
(Camera feed aata rahega)
('q' key dabao — window band ho jayegi)
```

Code	Kya karta hai?
<code>cap = VideoCapture(0)</code>	Laptop webcam (index 0)
<code>cap = VideoCapture(1)</code>	Baahri / USB camera (index 1)
<code>cap = VideoCapture(2)</code>	Teesra camera — Jetson pe koi bhi lane
<code>cap.isOpened()</code>	Camera mila ya nahi — True/False
<code>ret, frame = cap.read()</code>	ret = success, frame = image array
<code>cap.release()</code>	Camera properly band karna — zaroor likhna!

Try it yourself:

Upar wala live loop code type karo.
Run karo — camera window dikhi?
'q' dabao — band hua?
VideoCapture(0) se (1) karo — kya hota hai?

Topic 2 — Frame pe Shapes Draw karna

Real life analogy

Traffic system mein YOLO ek gaadi detect karta hai — uske around ek box draw karna padta hai. Yahi bounding box hota hai — `cv2.rectangle()` se draw hota hai.
Colour hamesha BGR format mein — Red = (0, 0, 255), Green = (0, 255, 0), Blue = (255, 0, 0).

Rectangle, Circle, Line — teeno ek saath:

```
import cv2

import numpy as np

# Blank image banana (kale background pe draw karenge)

# np.zeros((height, width, channels), dtype)
```

```

canvas = np.zeros((480, 640, 3), dtype='uint8')

# Rectangle — bounding box jaise
# cv2.rectangle(image, top-left, bottom-right, color_BGR, thickness)

cv2.rectangle(canvas, (50, 50), (250, 200), (0, 255, 0), 2)    # Green box

# Circle — detection center mark karne ke liye
# cv2.circle(image, center, radius, color_BGR, thickness)

cv2.circle(canvas, (400, 150), 60, (0, 0, 255), 3)            # Red circle

# Line — lane boundary dikhane ke liye
# cv2.line(image, start_point, end_point, color_BGR, thickness)

cv2.line(canvas, (0, 300), (640, 300), (255, 255, 0), 2)    # Yellow line

cv2.imshow('Drawing', canvas)

cv2.waitKey(0)

cv2.destroyAllWindows()

```

BGR Color Tuple	Traffic System mein use
(0, 255, 0)	Green — vehicle detected
(0, 0, 255)	Red — stop signal / alert
(255, 255, 0)	Cyan — lane boundary
(0, 255, 255)	Yellow — warning
(255, 255, 255)	White — text background
(0, 0, 0)	Black — blank canvas

Try it yourself:

Blank canvas pe teen rectangles draw karo — teen alag colors.

Ek circle beech mein draw karo.

Ek horizontal line canvas ke aadhe mein draw karo.

Topic 3 — Frame pe Text Likhna (putText)

Real life analogy

Traffic system mein detected vehicles ka count screen pe dikhana hota hai.

Jaise CCTV monitor pe 'North: 12 vehicles' likhta dikhta hai —

cv2.putText() se hum yahi karte hain: live frame pe text overlay karte hain.

putText syntax — parameters samjho:

```
import cv2

import numpy as np

canvas = np.zeros((480, 640, 3), dtype='uint8')

# cv2.putText(image, text, position, font, scale, color, thickness)

cv2.putText(

    canvas,

    'North Lane: 12 Vehicles',    # Text

    (30, 60),                    # Position (x, y) – top-left se

    cv2.FONT_HERSHEY_SIMPLEX,    # Font style

    0.8,                         # Font scale (size)

    (0, 255, 0),                 # Color – Green (BGR)

    2                            # Thickness

)

# Multiple lines — alag alag y position
```

```
lanes = [('North', 12), ('South', 5), ('East', 20), ('West', 8)]

for i, (lane, count) in enumerate(lanes):

    y_pos = 120 + i * 50          # Har line 50px neeche

    text = f'{lane}: {count} vehicles'

    cv2.putText(canvas, text, (30, y_pos),

                  cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

cv2.imshow('Lane Counter', canvas)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Font Style	Kab use karein?
<code>FONT_HERSHEY_SIMPLEX</code>	Sabse common — clean aur simple
<code>FONT_HERSHEY_DUPLEX</code>	Thoda bold — headings ke liye
<code>FONT_HERSHEY_COMPLEX</code>	Italic style
<code>FONT_HERSHEY_PLAIN</code>	Chhota aur simple

Try it yourself:

Blank canvas pe apna naam likhne wala code banao.
 4 lines mein 4 alag lanes aur counts likhne ki koshish karo.
 Ek line ka color green, doosri red karo.

Topic 4 — waitKey aur Keyboard Controls

Real life analogy

waitKey = system ko thoda rukne dena aur key check karna.
 Jaise security guard har 1 second mein check karta hai — koi alarm button daba to nahi?
 waitKey(1) = 1ms wait karo aur key check karo (live video ke liye)
 waitKey(0) = koi bhi key dabaane tak rokho (image display ke liye)

Multiple keys handle karna:

```
import cv2

cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    if not ret: break

    cv2.imshow('Camera', frame)

    key = cv2.waitKey(1) & 0xFF    # & 0xFF = safe way to read key

    if key == ord('q'):            # 'q' = quit

        print('Quit!')

        break

    elif key == ord('s'):          # 's' = screenshot save karo

        cv2.imwrite('screenshot.jpg', frame)

        print('Screenshot saved!')

    elif key == ord('g'):          # 'g' = grayscale toggle

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        cv2.imshow('Gray', gray)

cap.release()

cv2.destroyAllWindows()
```


& 0xFF kyun likhte hain?

Kuch systems mein waitKey() bada number return karta hai.

& 0xFF se hum sirf last 8 bits lete hain — jo actual key code hota hai.

Yeh ek safe practice hai — hamesha aise hi likhna.

Try it yourself:

Live loop mein 's' key se screenshot save karo.

Folder mein jao — screenshot.jpg bana?

'q' se band karo.

Mini Project: Live Traffic Monitor — Webcam pe Drawing + Text

Sab topics ek saath — live webcam, rectangle, text overlay, aur keyboard controls:

```
# Day 4 Mini Project — Live Traffic Monitor
```

```
import cv2
```

```
cap = cv2.VideoCapture(0)
```

```
# Simulated lane counts (baad mein YOLO se aayenge!)
```

```
lane_counts = {'North': 12, 'South': 5, 'East': 20, 'West': 8}
```

```
frame_number = 0
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    frame_number += 1
```

```
# === Drawing zone ===
```

Title bar — top pe blue rectangle

```
cv2.rectangle(frame, (0, 0), (640, 40), (180, 95, 24), -1) # -1 = filled
```

```
cv2.putText(frame, 'SMART TRAFFIC MONITOR', (10, 28),
```

```
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
```

Lane counts dikhao

```
for i, (lane, count) in enumerate(lane_counts.items()):
```

```
    y = 80 + i * 35
```

```
    color = (0, 255, 0) if count > 10 else (0, 200, 255)
```

```
    cv2.putText(frame, f'{lane}: {count} vehicles',
```

```
                (10, y), cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
```

Frame counter — bottom right

```
cv2.putText(frame, f'Frame: {frame_number}', (480, 460),
```

```
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (150, 150, 150), 1)
```

Busiest lane highlight karo

```
busiest = max(lane_counts, key=lane_counts.get)
```

```
cv2.putText(frame, f'Busiest: {busiest}!', (10, 240),
```

```
cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
```

```
cv2.imshow('Traffic Monitor', frame)
```

```
key = cv2.waitKey(1) & 0xFF
```

```
if key == ord('q'):
```

```
    break
```

```
elif key == ord('s'):

    cv2.imwrite('monitor_screenshot.jpg', frame)

    print('Screenshot saved!')

cap.release()

cv2.destroyAllWindows()
```

Yeh kahan kaam aayega?

Day 6-7 mein: lane_counts real YOLO detection se aayenge — yeh code waise ka waisa kaam karega!

Day 15 mein: 4 cameras ke liye 4 alag VideoCapture objects honge.

Abhi counts hardcoded hain — ek random.randint(0, 25) se counts badlo aur dekho live update.

Ghar pe challenge (optional)

lane_counts ek dictionary hai — 'q' dabane se pehle ek lane ka count badlao mid-loop.

Hint: cv2.waitKey ke baad lane_counts['North'] += 1 likho — counter live badhega!

Advanced: 's' key se screenshot aur uska frame number bhi filename mein daalo.

Hint: cv2.imwrite(f'frame_{frame_number}.jpg', frame)